

Raport: Analiza modeli predykcji prądów silników

Projekt: PUTM_EV_FF_CurrentLoop — Power Limiter / Torque Vectoring

Data: 2026-06-30

Autor: Michał Błotniak

Część I — Szczegółowe wyniki modeli

Zbiór danych

Parametr	Wartość
Plik	training_data.csv
Próbkowanie	5 Hz (co 200 ms)
Zbiór treningowy	Runy 2–8 → 3 730 próbek
Zbiór testowy	Runy 9–11 → 1 903 próbki
Jednostki prądu	Surowe ADC (a.u.) — przelicznik na Ampery nieznany
Zdarzenia > 80 kW w teście	1 903 (wszystkie próbki testowe)

FN (False Negative) = liczba zdarzeń wysokiej mocy, których model nie wykrył (niedoszacował prąd).
Niższe FN = bezpieczniejszy model dla limitera.

1. ARX — Regresja Ridge (wariant standardowy)

Wyniki liczbowe

Metryka	Wartość
R^2	0.569
RMSE	5 816 a.u.
MAE	2 364 a.u.
MedAE	367 a.u. ← najniższe ze wszystkich modeli
MAPE	85.1%
Próbki niedoszacowane	27.7%
Średnia wielkość niedoszacowania	4 094 a.u.
FN (przeoczone zdarzenia)	20
FP (fałszywe alarmy)	0
Recall	98.9%
Czas treningu	2.4 ms
Czas inferencji	0.0002 ms/próbkę (≈ 6 mln Hz)

R^2 per silnik

FL	FR	RL	RR
0.582	0.576	0.541	0.579

Interpretacja

ARX wyjaśnia $\sim 57\%$ wariancji prądu. MedAE = 367 a.u. jest najlepszy ze wszystkich testowanych modeli — **typowy błąd predykcji jest bardzo mały**. Problem leży w ogonach rozkładu: RMSE (5 816) jest $16\times$ wyższe niż MedAE, co oznacza nieliczne, ale duże błędy w momentach gwałtownych zmian prądu (transjenty).

FN = 20 oznacza, że z 1 903 zdarzeń wysokiej mocy 20 (1.1%) zostało przeoczonych — potencjalnie niebezpieczne dla limitera. Silnik RL ma najniższy R^2 (0.541) i powtarza ten wzorec we wszystkich modelach. Czas inferencji **0.0002 ms = 5 milionów predykcji na sekundę** — zdecydowanie najszybszy model. Port do C to jedno mnożenie macierzy ($31 \text{ wag} \times 4 \text{ silniki} = 124$ floaty).

2. ARX Conservative — Regresja kwantylowa $Q = 0.90$

Wyniki liczbowe

Metryka	Wartość
R^2	0.369
RMSE	7 040 a.u.
MAE	2 917 a.u.
MedAE	497 a.u.
MAPE	111.0%
Próbki niedoszacowane	14.2% (spadek z 27.7%)
Średnia wielkość niedoszacowania	5 001 a.u.
FN (przeoczone zdarzenia)	1 (spadek z 20)
FP (fałszywe alarmy)	0
Recall	99.9%
Czas treningu	3 210 ms
Czas inferencji	0.001 ms/próbkę ($\approx$ 955 tys. Hz)

R^2 per silnik

FL	FR	RL	RR
0.358	0.409	0.338	0.358

Interpretacja

Model przewiduje **90. percentyl** prądu zamiast średniej — w 90% przypadków rzeczywisty prąd jest poniżej predykcji, więc limiter reaguje ostrożniej. FN spadł z 20 → **1** (redukcja o 95%). Jedyna przeoczona próbka to prawdopodobnie ekstremalny transient.

R^2 spada do 0.369, a RMSE rośnie do 7 040 — model celowo zawyża, więc błędy mierzone względem wartości rzeczywistej są większe. To **pożądany efekt**: model jest konserwatywny. Niedoszacowania spadły z 27.7% → 14.2%, potwierdzając, że model rzadziej "opada poniżej" rzeczywistego prądu.

Czas treningu 3.2 s (vs. 2.4 ms dla Ridge) — QuantileRegressor używa solvera LP. Dla wdrożenia na VCU liczy się tylko inferencja (0.001 ms) — szybka.

3. ARMAX — ARX z ruchomą średnią residuów

Wyniki liczbowe

Metryka	Wartość
R^2	0.475
RMSE	6 423 a.u.
MAE	2 689 a.u.
MedAE	514 a.u.
MAPE	106.0%
Próbki niedoszacowane	31.1%
Średnia wielkość niedoszacowania	4 199 a.u.
FN (przeoczone zdarzenia)	69 ← najgorszy wynik
FP (fałszywe alarmy)	0
Recall	96.4%
Czas treningu	24.1 ms
Czas inferencji	0.232 ms/próbkę (4 302 Hz)

R^2 per silnik

FL	FR	RL	RR
0.619	0.517	0.333	0.585

Interpretacja

ARMAX jest gorszy od ARX pod **każdym** kluczowym względem: R^2 niższy (0.475 vs. 0.569), FN 3.5× wyższy (69 vs. 20), inferencja 1 000× wolniejsza (0.232 ms vs. 0.0002 ms). Szczególnie niepokojący jest $R^2_{RL} = 0.333$ — model prawie nie działa dla silnika tylnego lewego. Duża rozpiętość wyników per silnik (0.333–0.619) sugeruje, że bufor residuów MA wprowadza niestabilność.

ARMAX odrzucony — gorszy od ARX we wszystkich metrykach przy ogromnym koszcie czasu inferencji.

4. XGBoost — wariant standardowy (cel: średnia)

Wyniki liczbowe

Metryka	Wartość
R ²	0.589 ← najwyższe spośród wszystkich modeli
RMSE	5 683 a.u. ← najniższe spośród wszystkich modeli
MAE	2 502 a.u.
MedAE	749 a.u.
MAPE	100.7%
Próbki niedoszacowane	33.2%
Średnia wielkość niedoszacowania	3 846 a.u.
FN (przeoczone zdarzenia)	16
FP (fałszywe alarmy)	0
Recall	99.2%
Czas tuningu (RandomizedSearchCV)	8.9 s
Czas treningu (4 modele po tuningu)	1.4 s
Czas inferencji	0.053 ms/próbkę (18 761 Hz)

Najlepsze hiperparametry (tuning na FL, reużyte dla FR/RL/RR)

Parametr	Wartość
n_estimators	200
max_depth	3
learning_rate	0.1
subsample	0.7
colsample_bytree	0.7
min_child_weight	1

R² per silnik

FL	FR	RL	RR
0.590	0.601	0.579	0.573

Interpretacja

XGBoost ma najwyższe R² i najniższe RMSE — globalnie najlepsza dokładność. Wyniki per silnik są bardzo równomierne (0.573–0.601), co oznacza, że model nie faworyzuje żadnego silnika kosztem innego.

FN = 16 — nieznacznie lepszy niż ARX (20), ale niezerowy. 33.2% niedoszacowań to wysoki odsetek — model "upada" w tych samych gwałtownych transjentach co ARX. Czas inferencji 0.053 ms = ~18 700 predykcji/s — wystarczający dla 5 Hz, ale port do C/embedded jest trudniejszy (4 drzewa decyzyjne vs. 4 × wektor wag).

5. XGBoost Conservative — Regresja kwantylowa $Q = 0.90$

Wyniki liczbowe

Metryka	Wartość
R^2	0.431
RMSE	6 685 a.u.
MAE	3 258 a.u.
MedAE	1 122 a.u.
MAPE	175.5%
Próbki niedoszacowane	13.1% (spadek z 33.2%)
Średnia wielkość niedoszacowania	4 006 a.u.
FN (przeoczone zdarzenia)	0 ← zero!
FP (fałszywe alarmy)	0
Recall	100.0%
Czas treningu	2.0 s
Czas inferencji	~0.053 ms/próbkę

R^2 per silnik

FL	FR	RL	RR
0.452	0.462	0.416	0.396

Interpretacja

FN = 0 — model nie przeoczył żadnego zdarzenia wysokiej mocy w zbiorze testowym. To identyczny wynik co EKF (który był dotąd "bezpiecznym" rekordzistą), ale przy wyższym R^2 (0.431 vs. 0.416) i czasie treningu 2 s zamiast iteracyjnego dopasowywania parametrów fizycznych.

Niedoszacowania spadły z 33.2% → 13.1% — model zawyża prąd w ~87% próbek. Koszt: wyższe RMSE (6 685 vs. 5 683) i MedAE (1 122 vs. 749) — model jest bardziej agresywny w zawyżaniu przy spokojnej jeździe, co może powodować niepotrzebne ograniczenie momentu.

Zestawienie zbiorcze

Model	R ²	RMSE [a.u.]	MedAE [a.u.]	FN	Recall	ms/próbkę
XGBoost mean	0.589	5 683	749	16	99.2%	0.053
ARX Ridge	0.569	5 816	367	20	98.9%	0.0002
ARMAX	0.475	6 423	514	69	96.4%	0.232
XGBoost Q=0.90	0.431	6 685	1 122	0	100%	0.053
ARX Q=0.90	0.369	7 040	497	1	99.9%	0.001

Część II — Jak działają poszczególne modele i ich założenia

1. ARX — AutoRegressive eXogenous

Idea ogólna

ARX to model liniowy — predykcja jest **ważoną sumą** poprzednich wartości prądu i wejść sterujących (setpoint momentu, napięcie ogniwa). Nie ma czarnych skrzynek, nieliniowości ani pamięci ukrytej — jest to dosłownie jedno mnożenie wektora przez macierz i dodanie biasu.

Równanie

$$\begin{aligned} I_m(t+1) = & a1 \cdot I_m(t) + a2 \cdot I_m(t-1) + \dots + a5 \cdot I_m(t-4) \\ & + b1 \cdot T_sum(t) + b2 \cdot T_sum(t-1) + \dots + b5 \cdot T_sum(t-4) \\ & + b_FL \cdot I_FL(t) + b_FR \cdot I_FR(t) + b_RL \cdot I_RL(t) + b_RR \cdot I_RR(t) [+ \\ \text{lagged}] & \\ & + c \cdot U_dc(t) + \text{bias} \end{aligned}$$

Łącznie 31 cech: 4 silniki × 6 wartości (lag 0–5) + T_sum × 6 + U_dc × 1.

Sposób treningu

Metoda: **regresja Ridge** (least squares z regularyzacją L2). Rozwiązanie analityczne:

$$w = (X^T X + \alpha \cdot I)^{-1} X^T y$$

Jeden krok macierzowy — szybki i deterministyczny. Parametr `alpha=1.0` zapobiega przetrenowaniu na szumie. Dla 4 silników jednocześnie: y ma kształt (3730, 4), Ridge rozwiązuje 4 układy równań jednym wywołaniem.

Założenia

1. **Liniowość** — zależność prądu od cech jest liniowa. Przy uśrednionej dynamice 5 Hz jest to rozsądne przybliżenie.
2. **Stacjonarność** — parametry modelu nie zmieniają się w czasie (jeden zestaw wag dla całego przejazdu).
3. **Brak autokorelacji residuów** — błędy predykcji są traktowane jako biały szum.
4. **Cechy wystarczające** — prąd(t) + moment(t) + $U_{dc}(t)$ zawierają wystarczającą informację do predykcji $I(t+1)$.

Wariant Conservative $Q = 0.90$

Zamiast Ridge używamy QuantileRegressor — minimalizuje asymetryczną funkcję straty:

$$L(e) = 0.90 \cdot \max(e, 0) + 0.10 \cdot \max(-e, 0)$$

Kara za niedoszacowanie jest 9× większa niż za przeszacowanie. Wagi dobrane tak, żeby 90% czasu predykcja była powyżej prawdziwej wartości. Wymaga 4 osobnych modeli (jeden na silnik) — QuantileRegressor obsługuje tylko jeden output na raz.

2. ARMAX — ARX z ruchomą średnią residuów

Idea ogólna

ARMAX rozszerza ARX o **bufor ostatnich błędów predykcji**. Hipoteza: jeśli w poprzednim kroku model pomylił się o +1000 a.u., to prawdopodobnie w następnym też będzie blisko zera, więc historia błędów powinna poprawiać predykcję.

Równanie

$$I_m(t+1) = ARX(t) + c1 \cdot e_m(t) + c2 \cdot e_m(t-1) + c3 \cdot e_m(t-2) + bias$$

$$gdzie: e_m(t) = I_m(t) - \hat{I}_m(t) \quad [residuum \text{ z poprzedniego kroku}]$$

Sposób treningu (dwupasowy)

1. **Pas 1:** Wytrenuj ARX → oblicz residua treningowe na zbiorze train.
2. **Pas 2:** Rozszerz macierz cech o $N_{MA}=3$ ostatnich residuów → wytrenuj Ridge na rozszerzonej macierzy.

Ważna pułapka przy inferencji: residua muszą być akumulowane krok po kroku — każda predykcja wymaga znajomości poprzedniej predykcji i rzeczywistej wartości. Inferencja jest sekwencyjna, co powoduje że jest 1000× wolniejsza niż ARX.

Założenia

1. **Autokorelacja błędów** — zakłada się, że residua są autokorelowane i ich historia poprawia predykcję. Przy 5 Hz ta autokorelacja jest słaba — stąd słabe wyniki.
2. **Stabilność pętli feedbacku** — bufor residuów tworzy pętlę sprzężenia zwrotnego; przy pewnych wartościach wag może prowadzić do dryfu lub oscylacji.
3. Wszystkie założenia ARX plus powyższe.

Dlaczego gorszy niż ARX przy naszych danych?

Przy 5 Hz prąd zmienia się na tyle dynamicznie, że residuum w kroku t niesie mało informacji o residuum w kroku $t+1$. Bufor błędów zamiast poprawiać — wprowadza dodatkowy szum, który obniża recall i podnosi FN z 20 (ARX) do 69 (ARMAX).

3. XGBoost — Gradient Boosted Trees

Idea ogólna

XGBoost buduje **sekwencję drzew decyzyjnych**, gdzie każde drzewo koryguje błąd poprzedniego. Na końcu predykcja to ważona suma wyników wszystkich 200 drzew. Model może uchwycić nieliniowe zależności, interakcje między cechami i skokowe przejścia — dlatego osiąga wyższe R^2 niż ARX.

Równanie (uproszczone)

$$\hat{I}_m(t+1) = \sum_{k=1}^{200} \eta \cdot \text{tree}_k(\text{cechy}(t))$$

gdzie $\eta = 0.1$ (learning rate), tree_k to k -te drzewo o $\text{max_depth} = 3$

Sposób treningu

1. Inicjalizacja: predykcja = średnia z y_{train} .
2. Iteracja $k = 1 \dots 200$:

3. Oblicz gradienty (pochodne funkcji straty względem aktualnej predykcji).
4. Dopasuj nowe drzewo minimalizując te gradienty.
5. Dodaj drzewo z wagą η do sumy.
6. **RandomizedSearchCV** na silniku FL: 20 losowych kombinacji hiperparametrów, 3-fold CV, minimalizacja RMSE.
7. Najlepsze parametry reużyte dla FR, RL, RR.

Kluczowe hiperparametry i ich rola:

- `max_depth=3` — płytkie drzewa, mniejsze ryzyko przetrenowania
- `subsample=0.7` — każde drzewo trenowane na 70% próbek (stochastyczność = regularyzacja)
- `colsample_bytree=0.7` — każde drzewo widzi losowe 70% cech

Założenia

1. **Brak założeń o liniowości** — może nauczyć się dowolnej funkcji przybliżalnej podziałami osi.
2. **I.I.D.** — próbki treningowe traktowane jako niezależne (brak jawnego modelowania kolejności czasowej).
3. **Szeregowość przez lag features** — informacja historyczna zakodowana przez lag features (do 5 kroków), nie ma explicite pamięci jak w LSTM/ESN.
4. **Stabilność dystrybucji** — drzewa nie adaptują się online; zakłada się, że rozkład testowy (runy 9–11) jest podobny do treningowego (runy 2–8).

Wariant Conservative Q = 0.90

Zmiana funkcji straty na `reg:quantileerror` z `quantile_alpha=0.90`:

$$L(e) = 0.90 \cdot \max(e, 0) + 0.10 \cdot \max(-e, 0)$$

Drzewa są nadal budowane sekwencyjnie, ale gradienty liczone z tej asymetrycznej straty zamiast kwadratowej. Architektura i czas inferencji identyczne jak wariant standardowy. Wynik: FN = 0, koszt = wyższe RMSE i bardziej zawyżone predykcje przy spokojnej jeździe.

Porównanie założeń i właściwości

Cecha	ARX	ARMAX	XGBoost
Typ modelu	Liniowy	Liniowy + feedback	Nieliniowy (drzewa)
Pamięć czasowa	Lag features (explicite)	Lag features + residua	Lag features (explicite)
Nieliniowość	Brak	Brak	Pełna
Założenie o rozkładzie	Brak	Brak	Brak
Autokorelacja residuów	Ignorowana	Modelowana (MA)	Ignorowana
Port do C	Trivialny (31 floatów)	Trudny (seq. feedback)	Trudny (drzewa)
Adaptowalność online	Nie	Nie	Nie
Sensowny przy małym zbiorze	Tak	Tak	Granicznie (3 730 próbek)
Wariant konserwatywny	QuantileRegressor	—	<code>reg:quantileerror</code>

Rekomendacja wdrożeniowa

Zastosowanie	Model	Uzasadnienie
Safety-critical limiter	XGBoost Q=0.90	FN=0, Recall=100%, żadne zdarzenie nieprzeoczone
Safety-critical + łatwy port C	ARX Q=0.90	FN=1, inferencja 0.001 ms, 31 floatów na silnik
Hybrydowy (accuracy + safety)	ARX mean + XGBoost Q=0.90	ARX jako szybki predyktor, XGBoost Q jako filtr bezpieczeństwa
Odrzucone	ARMAX	Gorszy od ARX w każdej metryce, 1000× wolniejszy

Proponowane podejście na VCU:

1. **ARX Q=0.90** jako główny predyktor — szybki, trivialny port do C, FN=1

2. **XGBoost Q=0.90** jako drugi głos — jeśli oba modele sygnalizują > 80 kW, ogranicz moment niezależnie od siebie
 3. **EKF** jako fizyczny filtr bezpieczeństwa — FN=0, konserwatywne przeszacowanie wbudowane w równania stanu
-

Wygenerowano na podstawie wyników notebooków: `03_xgboost_current`, `04_arx_armax`.